

BARAJ DOLULUK TAHMİN RAPORU

Mehmet Kordon
221307022
mehmetkordon@kocaeli.edu.tr

Hakan Aytuğ Fırat
22130703015
hakanaytugfirat@kocaeli.edu.tr

I. GİRİŞ

Su kaynaklarının yönetimi, büyük metropoller için hayati bir öneme sahiptir. İstanbul gibi nüfusu her geçen gün artan ve iklim değişikliklerinin etkisi altında olan bir şehirde, su kaynaklarını verimli kullanmak ve planlamak kritik bir gerekliliktir. İstanbul'un su ihtiyacı büyük oranda barajlar ve rezervuarlar aracılığıyla karşılanmaktadır.

Büyükçekmece, Istrancalar, Kazandere, Pabuçdere, Sazlıdere, Alibey, Ömerli, Termos, Darlık ve Elmalı gibi şehrin önde gelen 10 barajının su doluluk oranlarının doğru bir şekilde tahmin edilmesi, hem su kaynaklarının sürdürülebilir yönetimi hem de kuraklık risklerinin önceden belirlenerek gerekli adımların atılması için hayati bir rol oynamaktadır. Bu çalışma, İstanbul'daki barajların gelecekteki doluluk oranlarını tahmin etmeyi amaçlamaktadır. Tarihsel su seviyeleri, yağış miktarları, sıcaklık ve zaman faktörleri gibi veriler kullanılarak doğru ve kapsamlı bir tahmin modeli geliştirilmesi hedeflenmiştir. Çalışmamızda FEDFORMER, ETSFORMER, ARITMA, INFORMER ve TFT modelleri kullanılmıştır.

II. MODELLER

A. ETSFORMER Modeli

ETSformer, zaman serisi tahmininde klasik yöntemlerle modern derin öğrenme yaklaşımlarını birleştiren bir modeldir. Model, **Üstel Düzgünleştirme (Exponential Smoothing)** ve **Dönüşümcü (Transformer)** yapılarının güçlü yönlerini bir araya getirir. Bu yaklaşım, özellikle sezonluk ve trend bileşenlerinin belirgin olduğu veri setlerinde etkili sonuçlar elde edilmesini sağlar. Baraj doluluk oranlarındaki mevsimsel ve trend değişikliklerini modellemek için ETSformer, uzun vadeli bağımlılıkları yakalama ve daha doğru tahminler yapma kapasitesine sahiptir. Bu nedenle çalışmamızda kullanılan temel modellerden biri olarak seçilmiştir.

B. Temporal Fusion Transformer (TFT) Modeli

Temporal Fusion Transformer (TFT), zaman serisi tahminlerinde esnek ve güçlü bir modeldir. Bu model, geçmiş verilerdeki trendleri ve mevsimsel desenleri anlamak için dikkat mekanizmasını kullanır. Baraj doluluk oranlarındaki değişkenliği tahmin etmek için uygun olan TFT, su seviyesi tahminlerinde daha hassas sonuçlar elde edilmesini sağlar. Ayrıca, veriler arasındaki karmaşık ilişkileri çözümleyerek tahmin doğruluğunu artırır.

C. INFORMER Modeli

Informer, uzun vadeli bağımlılıkları ve büyük ölçekli zaman serisi verilerini işlemek için geliştirilmiş bir modeldir. Özellikle baraj doluluk oranlarındaki mevsimsel ve zamansal desenleri anlamada etkili olan Informer, dikkat mekanizmasını optimize ederek daha

hızlı ve doğru tahminler yapar. Yüksek verimli yapısıyla büyük veri setlerinde dahi güçlü performans gösterir ve tahmin sürecini hızlandırır.

D. FEDFORMER Modeli

FEDformer (Fourier Enhanced Decomposition Transformer), zaman serisi tahminlerinde yüksek doğruluk sağlamak için geliştirilmiş yenilikçi bir modeldir. Bu model, zaman serisi verilerini sezonsal ve trend bileşenlerine ayırarak daha net bir analiz yapar. Fourier dönüşümleriyle uzun vadeli bağımlılıkları etkili bir şekilde modelleyen FEDformer, özellikle baraj doluluk oranları gibi karmaşık ve büyük ölçekli veri setlerinde güçlü performans sunar. Düşük hesaplama maliyetiyle öne çıkan bu model, hızlı ve doğru tahminler için idealdir.

E. ARIMA Modeli

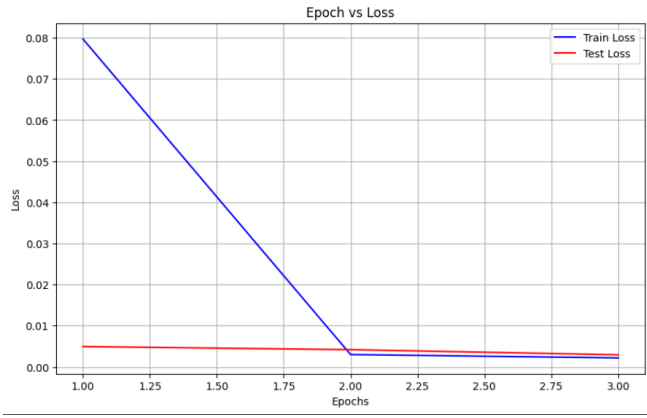
ARIMA (Autoregressive Integrated Moving Average), zaman serisi tahmininde sıkça kullanılan klasik bir modeldir. Bu model, geçmiş verilere dayalı olarak gelecek değerleri tahmin etmek için otoregresyon (AR), fark alma (I), ve hareketli ortalama (MA) bileşenlerini birleştirir. Baraj doluluk oranlarındaki trendleri ve kısa vadeli değişiklikleri yakalamada etkili olan ARIMA, özellikle istasyonier (sabit ortalamalı ve varyanslı) verilerde yüksek doğruluk sağlar. Basit yapısı ve yorumlanabilir sonuçlarıyla, modern modeller için bir karşılaştırma noktası olarak da kullanılır.

III. MODELLERİMİZİN EĞİTİMİ

A. Bu bölümde, baraj doluluk oranlarını tahmin etmek için kullanılan modellerin eğitim süreci detaylandırılmaktadır. Modeller, sıcaklık, yağış, basınç, nem, bulut örtüsü, ortalama rüzgar hızı, gün ışığı süresi ve geçmiş baraj doluluğu verilerini dikkate alınmıştır. Test-Train verilerinin epoch/loss grafiğini çizdirirken MSE, MAPE, MAPE, RMSE, R^2 , Training time ve Inference time değerlerini hesapladık.

B. ETSFORMER Modelinin uygulması

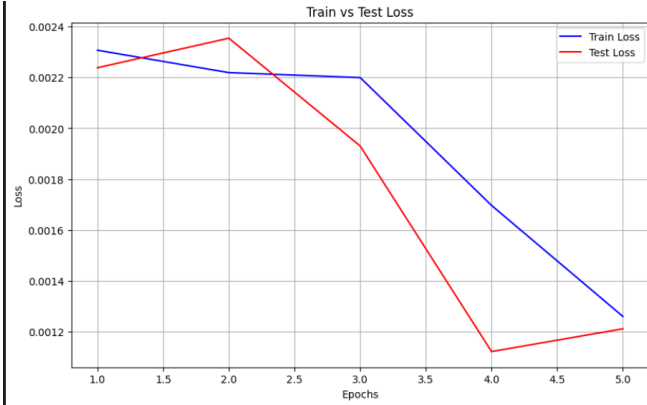
Örnek olarak Kazandere barajı için kullandığımız kodların github bağlantısından bulabilirken ve bazı sonuç değerlerinin fotoğraflarını eke koyuyorum.



```
Epoch 1/3, Train Loss: 0.07978586990402942, Test Loss: 0.004972218875596545
Train MSE: 0.0801517459366114, Test MSE: 0.004202763185894934
Train MAPE: 113.82843255996704%, Test MAPE: inf%
Train MAE: 0.19536176671244437, Test MAE: 0.04512672535756714
Train RMSE: 0.28116838486469, Test RMSE: 0.06402872192088113
Train R²: 0.12698183098672295, Test R²: 0.95268264931264022
Training time for epoch 1: 45.30 seconds
Inference time for epoch 1: 0.66 seconds
python-input-22-aab6dc8c6ba:157: RuntimeWarning: divide by zero encountered in divide
test_mape = np.mean(np.abs((np.array(test_actuals) - np.array(test_preds)) / np.array(test_actuals))) * 100)
Train MSE: 0.00298688431032791, Test MSE: 0.003949533462037773
Train MAPE: 18.07384498966797%, Test MAPE: inf%
Train MAE: 0.03717681644346045, Test MAE: 0.04574478071851608
Train RMSE: 0.0546780434896619, Test RMSE: 0.06284531376353987
Train R²: 0.9748961845995922, Test R²: 0.955533668790202
Training time for epoch 2: 11.75 seconds
Inference time for epoch 2: 0.53 seconds
python-input-22-aab6dc8c6ba:157: RuntimeWarning: divide by zero encountered in divide
test_mape = np.mean(np.abs((np.array(test_actuals) - np.array(test_preds)) / np.array(test_actuals))) * 100)
Epoch 3/3, Train Loss: 0.002175143804807796, Test Loss: 0.002926096892615736
Train MSE: 0.0021869728122498916, Test MSE: 0.002600071593214638
Train MAPE: 12.92661428451538%, Test MAPE: inf%
Train MAE: 0.0316648156847615, Test MAE: 0.0265571286034527
Train RMSE: 0.046765072567559345, Test RMSE: 0.05098089716832302
Train R²: 0.9816364337127956, Test R²: 0.978726759604443
Training time for epoch 3: 49.49 seconds
Inference time for epoch 3: 0.48 seconds
```

C. Temporal Fusion Transformer (TFT) Modeli

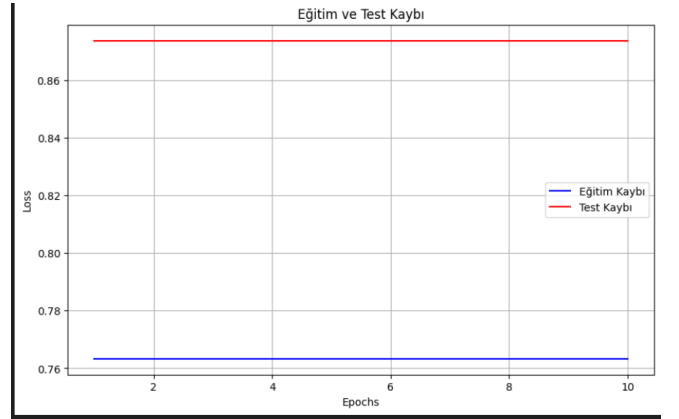
Bu kodda farklı olarak sıfıra bölme hatasını engellemek için küçük bir sabit kullanıyoruz ve gizli katman sayısını iyi sonuç almak için yükselttik. Kodları github bağlantımızda bulabilirsiniz.



```
Epoch 2/5, Train Loss: 0.0022190721913351004, Test Loss: 0.0023545978370641345
Train MSE: 0.00222263223000779, Test MSE: 0.0024081793994396556
Train MAPE: inf%, Test MAPE: 650.0773428670605%
Train MAE: 0.03904944544871236, Test MAE: 0.0174518004594848806
Train RMSE: 0.0474086575153805, Test RMSE: 0.0489028225819934
Train R²: 0.6121601599174624, Test R²: 0.10625232804805749
Training Time: 66.27942005266113 seconds, Inference Time: 6.875481843948364 seconds
python-input-8-0a6bea7ac18d:147: RuntimeWarning: divide by zero encountered in divide
train_mape = np.mean(np.abs((np.array(train_actuals) - np.array(train_preds)) / np.array(train_actuals))) * 100)
Epoch 3/5, Train Loss: 0.0021995117597442162, Test Loss: 0.001930124219999176
Train MSE: 0.0021954666913820615, Test MSE: 0.00197688191922879133
Train MAPE: inf%, Test MAPE: 293.3019396619443%
Train MAE: 0.03797379086252088, Test MAE: 0.028406716071748512
Train RMSE: 0.0468558074609732, Test RMSE: 0.04446143480780561
Train R²: 0.5927218176483062, Test R²: 0.0911489923980973
Training Time: 67.06980741325378 seconds, Inference Time: 8.04568099975586 seconds
python-input-8-0a6bea7ac18d:147: RuntimeWarning: invalid value encountered in divide
train_mape = np.mean(np.abs((np.array(train_actuals) - np.array(train_preds)) / np.array(train_actuals))) * 100)
Epoch 4/5, Train Loss: 0.0016963344450006148, Test Loss: 0.0011209631208176313
Train MSE: 0.0017021617033917077, Test MSE: 0.0011445043773571928
Train MAPE: nan%, Test MAPE: 133.40208338813707%
Train MAE: 0.029604948229931592, Test MAE: 0.0133292650412532
Train RMSE: 0.04125726243210651, Test RMSE: 0.03383052434351547
Train R²: 0.2348491952919912, Test R²: 0.47380071904308805
Training Time: 65.77826831129456 seconds, Inference Time: 7.408727169036865 seconds
python-input-8-0a6bea7ac18d:147: RuntimeWarning: invalid value encountered in divide
train_mape = np.mean(np.abs((np.array(train_actuals) - np.array(train_preds)) / np.array(train_actuals))) * 100)
Epoch 5/5, Train Loss: 0.0012552328052906, Test Loss: 0.001207234319152255
Train MSE: 0.00126301209608371935, Test MSE: 0.0012323809142965448
Train MAPE: nan%, Test MAPE: 123.24196100234985%
Train MAE: 0.01887542366561623, Test MAE: 0.01500809758739524
Train RMSE: 0.03553916284941436, Test RMSE: 0.03510539722450872
Train R²: 0.08372156967293531, Test R²: 0.4334033825726164
Training Time: 66.23243680003405 seconds, Inference Time: 6.08377799987793 seconds
```

D. ARIMA Modeli

Bu projede, zaman serisi verilerini kullanarak baraj doluluk oranını tahmin etmek için ARIMA modelini kullandık. İlk olarak, verileri sıraladık ve her bir bağımsız değişkenin ağırlıklarını belirleyerek ağırlıklı bir skor hesapladık. Ardından, eğitim ve test setlerine ayırarak, ARIMA modelini (10,1,5) (10 yıllık geçmiş verilerini birer yıl arayla geriye dönük 5 hata geçmiş ile ilgilenir) parametreleriyle eğittik. Modeli eğitirken, eğitim süresi hesaplandı ve modelin özetine göz atıldı. Test verisi üzerinde yapılan tahminleri görselleştirerek, modelin başarımını MSE, RMSE, MAE, MAPE ve R² metrikleriyle değerlendirdik



Mean Squared Error (MSE): 0.7633347101061798
Root Mean Squared Error (RMSE): 0.8736902827124609
Mean Absolute Error (MAE): 0.1874770647221426
Mean Absolute Percentage Error (MAPE): 1.4854573757249832%
R²: -0.011286224448584292
Eğitim Süreti: 12.680253505706787 saniye
Çıkarım Süreti: 0.06465768814086914 saniye

```
weights = {
    'temp': 0.4,
    'feelslike': 0.3,
    'dew': 0.05,
    'humidity': 0.1,
    'precip': 0.7,
    'windspeedmean': 0.05,
    'sealevelpressure': 0.1,
    'cloudcover': 0.05,
    'dakika': 0.12
}

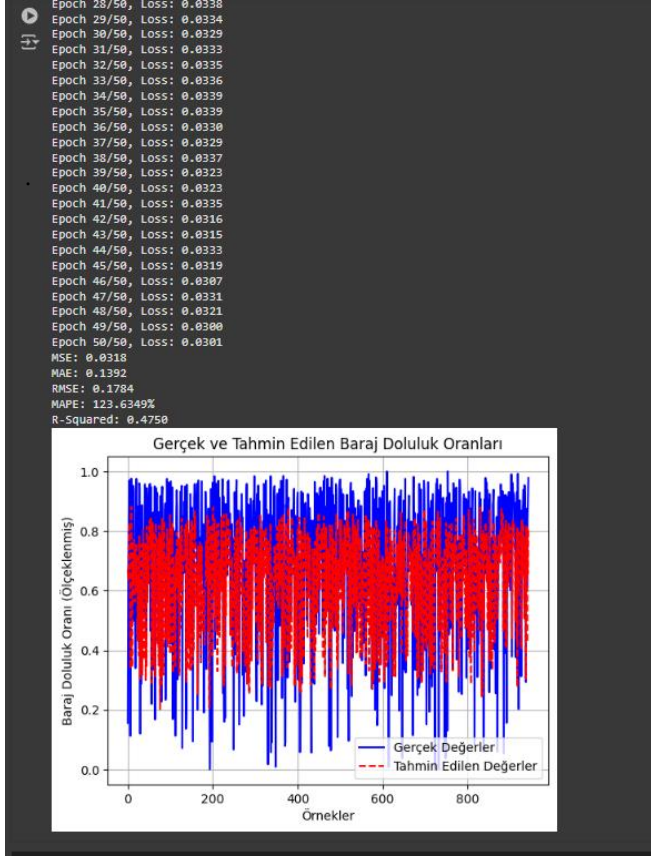
# Baraj doluluk yüzdesi ve diğer bağımsız değişkenler
df = df[['datetime', 'temp', 'feelslike', 'dew', 'humidity', 'precip',
        'windspeedmean', 'sealevelpressure', 'cloudcover', 'BarajDoluluk',
        'dakika']]
```

Figure 1 Deneme yanılmayla verdiğimiz ağırlık değerleri

E. FEDformer Modelinin uygulanışı

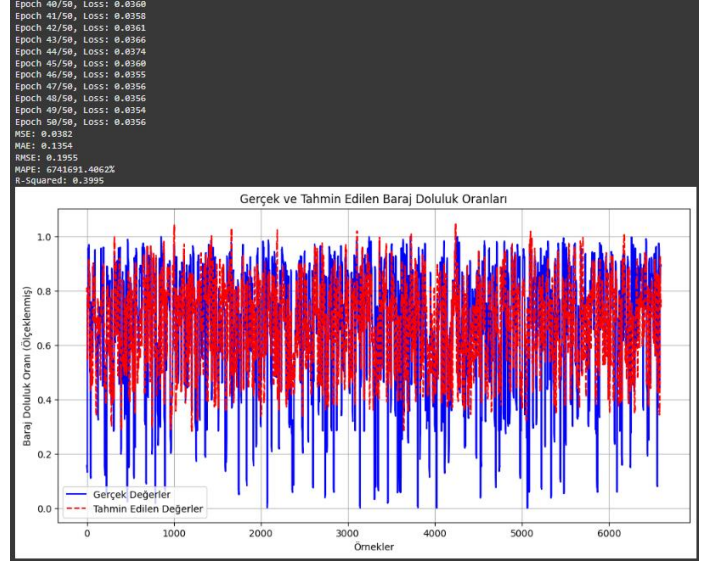
Bu projede, zaman serisi verilerini kullanarak baraj doluluk oranını tahmin etmek için FEDformer modelini kullandık.

FEDformer modelinin eğitim süreci boyunca kayıp değerinde belirgin bir azalma gözlemlenmiştir; ilk epoch'ta 0.1084 olan kayıp değeri, eğitim tamamlandığında 0.0301'e kadar düşmüştür. Modelin test sonuçları, ortalama hata (MAE: 0.1392) ve karekök hata (RMSE: 0.1784) açısından nispeten kabul edilebilir seviyededir. R-Squared metriği ise modelin veri varyansının yalnızca %47.5'ini açıklayabildiğini ortaya koymuştur.



F. Informer Modelinin Uygulanışı

Informer modeli, eğitim sürecinde başlangıçta 0.0635 olan kayıp değerini 50 epoch sonunda 0.0356'ya kadar düşürerek belirli bir iyileşme sağlamıştır. Test sonuçlarında MSE: 0.0382 ve MAE: 0.1354 değerleri modelin makul bir hata oranına sahip olduğunu göstermektedir. Ancak, RMSE'nin 0.1955 seviyesinde olması hata payının yüksek olduğunu açıklar, R-Squared değerinin 0.3995 olması, modelin veri varyansının yalnızca %39.95'ini açıklayabildiğini ortaya koymuştur.



Kaynakça

<https://github.com/lucidrains/ETSformer-pytorch>
<https://github.com/microsoft/ML-For-Beginners/blob/main/7-TimeSeries/2-ARIMA/README.md>
<https://www.visualcrossing.com/>
<https://paul-bruffett.medium.com/breaking-down-etsformer-aad0b8261465>
<https://medium.com/deep-learning-turkiye/derin-ogrenme-uygulamalarinda-en-sik-kullanilan-hiper-parametreler-ece8e9125c4>
<https://medium.com/@lotussavy/informer-beyond-efficient-transformer-for-long-sequence-time-series-forecasting-141488c6e22e>
https://medium-com.translate.goog/intel-tech/how-to-apply-transformers-to-time-series-models-spacetimeformer-e452f2825d2e?_x_tr_sl=en&_x_tr_tl=tr&_x_tr_hl=tr&_x_tr_pto=tc&_x_tr_hist=true
<https://github.com/hakan8755/barajDolulukTahmin>